Nombre del proyecto

Clasificación automática de normativa sobre inteligencia artificial en la educación superior de América Latina y el Caribe: hacia un Índice de Madurez Normativa (IMN) escalable

Entregable: Reporte técnico: Diagnóstico de Overfitting/Underfitting · Semana 3 · Proyecto Integrador en IA

Profesor	Materia	Fecha de Inicio	Fecha de Finalización
Gladys María Villegas Rugel	Proyecto Integrador	8 de septiembre de 2026	25 de octubre de 2026

Resumen del Proyecto

Reporte técnico de la actividad de la Semana 3: sistema de tracking de métricas por época, curvas de aprendizaje (por época, por tamaño de datos y por hiperparámetro), diagnóstico sistemático con criterios cuantitativos prefijados y cuatro estrategias de mejora evaluadas antes/después sobre el clasificador de normativa sobre IA del corpus GENIAL (regresión logística sobre TF-IDF).

Responsables del proyecto

Nombre	Rol	Tareas
Jorge Javier Maldonado Mahauad	Líder de las Fases 1, 2 y 4. Enlace con GENIAL.	Problema y estado del arte; depuración del corpus, variable idioma y muestreo; doble codificación a ciegas y cálculo de κ.
Melissa Figallo	Líder de las Fases 3 y 5.	Rama clásica y rama LLM few-shot con comparación pareada; ética, agregación al IMN, documentación y pitch.

Diagnóstico de overfitting y underfitting del clasificador de normativa sobre IA (corpus GENIAL)

Proyecto Integrador en IA (MIAR0545) · Semana 3 · Grupo 9 — Jorge Javier Maldonado Mahauad · Melissa Figallo · Tutora: Gladys María Villegas Rugel. Reporte técnico de la actividad «Diagnóstico de Overfitting/Underfitting»; acompaña al cuaderno ejecutado *overfitting_analysis.ipynb* y al módulo *src/genial_clf/tracking.py* del repositorio *github.com/jorgemaldonadom-cmd/genial-clasificador-normativa*.

1. Resumen ejecutivo

Se diagnosticó el comportamiento de entrenamiento del modelo del Proyecto Integrador —regresión logística sobre TF-IDF (1-2 gramas, 5 000 características, clases ponderadas) que asigna una de cinco dimensiones del framework GENIAL a cada norma— con un sistema de tracking por época, curvas de aprendizaje por época, por tamaño de datos y por hiperparámetro, criterios cuantitativos fijados de antemano y cuatro estrategias de mejora evaluadas con el mismo protocolo (validación cruzada estratificada por grupo de URL, 5 pliegues, 673 normas de entrenamiento; el conjunto de prueba a ciegas del proyecto no se utilizó).

Tabla 1. Hallazgos principales del diagnóstico.

Pregunta	Evidencia	Respuesta
¿Sobreajusta?	F1 macro de entrenamiento 0.94 frente a 0.55 en validación (gap 0.40); exactitud de entrenamiento 1.00 con validación ≈ 0.56; la pérdida de validación toca mínimo en la época 46 y rebota 2.5 % mientras la de entrenamiento sigue bajando	Sí: overfitting (varianza alta). Se cumplen las tres firmas de la actividad
¿Subajusta?	El entrenamiento se ajusta casi perfectamente (pérdida final 0.15); las curvas no convergen a valores altos de pérdida	No
¿Dónde se concentra?	Gap de F1 por clase: Técnico 0.67 y Administrativo 0.57 (clases con 50 y 100 ejemplos) frente a 0.13 en Protección de Datos	En las clases minoritarias
¿Ayudaría tener más datos?	La validación gana 17 pp de F1 entre n = 53 y n = 538 (3.8 pp por 100 normas en promedio) pero solo 1.0 pp por 100 normas en el último tramo	Sí, con rendimientos decrecientes
¿Qué lograron las mejoras?	Regularización fuerte + vocabulario reducido: gap 0.40 → 0.25, F1 de validación 0.547 → 0.519; en PyTorch, weight decay 0.05: gap de exactitud 0.44 → 0.25; early stopping en la época 46 sin pérdida de F1	Cierran el gap; la validación no mejora de forma significativa
¿Qué explica el techo?	F1 de validación casi plano (0.53-0.55) para C entre 0.01 y 100, mientras el de entrenamiento va de 0.84 a 1.00	Parte del gap es error irreducible (normas de frontera, etiqueta única), no varianza

Recomendación. Adoptar en el proyecto la configuración regularizada (menor varianza al mismo rendimiento) y el early stopping en los modelos entrenados por épocas; mantener la validación cruzada por grupo; y dirigir el esfuerzo de mejora a donde la evidencia señala retorno: más normas codificadas de las clases Técnico y Administrativo, representaciones con conocimiento externo (rama LLM few-shot del proyecto) y revisión de las etiquetas ambiguas identificadas por el diagnóstico de multi-etiqueta. Estas conclusiones se incorporan a la discusión de resultados del Proyecto Integrador.

2. Metodología utilizada

2.1 Datos y particiones

Se usó el conjunto de entrenamiento del proyecto (673 normas; distribución Pedagógico 246, Ética 219, Administrativo 100, Protección de Datos 58, Técnico 50; desbalance 4.9:1; mediana de 45 palabras por norma). Para las curvas por época se partió en entrenamiento (538) y validación (135) con *StratifiedGroupKFold*: estratificación por dimensión y grupo = URL oficial, de modo que las normas duplicadas no se repartan entre ambos lados. Para las curvas por tamaño, por hiperparámetro y para la comparación de estrategias se usó validación cruzada de 5 pliegues con el mismo esquema sobre las 673 normas. El conjunto de prueba a ciegas del proyecto (125 normas) no se tocó: su uso aquí contaminaría la evaluación final [1], [2].

2.2 Sistema de tracking

El módulo *tracking.py* implementa un *MetricsTracker* independiente del framework: cada época registra pérdida (log-loss), exactitud y F1 macro en entrenamiento y validación, con marca de tiempo y los hiperparámetros del *run*, y exporta CSV y JSON. Tres adaptadores lo alimentan: (a) *EpochTrainer* para scikit-learn, que entrena un *SGDClassifier* con pérdida logística época a época mediante *partial_fit* [3], [4]; (b) un bucle de entrenamiento manual en PyTorch [5] (regresión softmax sobre TF-IDF, entropía cruzada ponderada por clase, Adam [6]) con early stopping opcional; y (c) un *callback* de Keras (*on_epoch_end*) que se define pero no se ejecuta porque TensorFlow no está instalado en el entorno y el modelo del proyecto no es una red neuronal; la dinámica por épocas queda cubierta por (a) y (b). Los envoltorios *learning_curve_tracked* y *validation_curve_tracked* aplican las funciones homónimas de scikit-learn [7], [8] con validación por grupo. El tracker escribe archivos planos; su historial puede volcarse a MLflow [9] en una línea si se requiere un servidor de experimentos.

2.3 Curvas generadas

Curva	Qué varía	Modelo	Métrica
A · pérdida por época	épocas (1-150 PyTorch; 1-80 SGD)	regresión softmax PyTorch; SGD scikit-learn	log-loss entrenamiento y validación
B · exactitud / F1 por época	épocas	ídem	exactitud y F1 macro
C · aprendizaje por tamaño	8 tamaños del 10 % al 100 % del entrenamiento	regresión logística C = 1 (CV por grupo)	F1 macro ± DE entre pliegues
D · validación por hiperparámetro	C ∈ {0.01 … 100}; max_features ∈ {250 … 20 000}	regresión logística (CV por grupo)	F1 macro ± DE

Todas las figuras se generaron a 300 DPI con título, etiquetas de ejes, leyenda, colores contrastantes (azul = entrenamiento, naranja = validación), rejilla y anotaciones en los puntos críticos (mínimo de la pérdida de validación, máximo de validación, gap final, configuración base y mejor hiperparámetro).

2.4 Criterios de diagnóstico (fijados antes de mirar las curvas)

Siguiendo el marco sesgo-varianza [10]–[12], la función *diagnose_epochs* clasifica el comportamiento con umbrales explícitos: **overfitting** si el gap final de exactitud entrenamiento−validación supera 0.15 **o** la pérdida de validación sube al menos 5 % desde su mínimo mientras la de entrenamiento sigue bajando (el criterio de Prechelt para early stopping [13]); **underfitting** si la exactitud de entrenamiento final es inferior a 0.55 (ambas curvas convergen a un rendimiento pobre); **óptimo** en otro caso. La función *diagnose_size* calcula, sobre la curva por tamaño, el gap final, la pendiente de la validación (global y en el último tramo, en puntos porcentuales de F1 por cada 100 normas) y la desviación entre pliegues [14], [15]. El análisis cuantitativo se completa con el gap de F1 por clase en validación cruzada.

2.5 Protocolo de comparación de estrategias

Toda configuración se evalúa con el mismo esquema de validación cruzada (mismas particiones, misma semilla) y se reporta F1 macro de entrenamiento y validación, su desviación entre pliegues y el gap. Para elegir la fuerza de regularización se usó la regla de una desviación estándar [12]: el modelo más regularizado cuyo F1 de

validación esté a ≤ 1 DE del máximo, que evita elegir por ruido entre configuraciones empatadas. Herramientas: Python 3.11, scikit-learn 1.8, PyTorch 2.14 (CPU), pandas, matplotlib/seaborn; semilla 2026.

3. Resultados del diagnóstico

3.1 Modelo base

En la partición 80/20 el modelo del proyecto obtiene exactitud 0.954 y F1 macro 0.948 en entrenamiento frente a 0.615 y 0.551 en validación. En validación cruzada por grupo (5 pliegues): F1 macro de entrenamiento 0.942 ± 0.005 y de validación 0.547 ± 0.023, gap 0.395. (El F1 de validación es inferior al 0.587 reportado en el experimento del proyecto porque aquí se entrena con 673 normas en lugar de 813 y no se incluye el conjunto ciego.)

3.2 Identificación de patrones

Tabla 2. Contraste de las firmas de overfitting, underfitting y comportamiento óptimo con la evidencia observada.

Patrón	Evidencia observada	¿Se cumple?
OVERFITTING · gap creciente entre training y validation loss	PyTorch: gap de pérdida final 1.05 (train 0.15 vs val 1.20); SGD: 0.86	sí
OVERFITTING · training accuracy muy alta, validation accuracy estancada	PyTorch: train acc 1.00 vs val acc 0.56 (gap 0.44); SGD: 1.00 vs 0.59	sí
OVERFITTING · validation loss aumenta mientras training loss disminuye	PyTorch: val loss mínima en la época 46 y rebote de 2.5 % hasta la época 150	no
UNDERFITTING · ambas curvas convergen a valores altos de loss	train loss final 0.15: el entrenamiento se ajusta casi perfectamente	no
UNDERFITTING · training accuracy baja y no mejora	train acc final 1.00	no
UNDERFITTING · gap pequeño pero rendimiento pobre	gap 0.44: no es pequeño	no
ÓPTIMO · gap pequeño y estable, curvas convergen, rendimiento satisfactorio	gap F1 en CV 0.40; val F1 0.55 < meta 0.70	no
CURVA POR TAMAÑO · validación aún creciente con más datos	pendiente +3.8 pp de F1 por 100 normas (global), +1.0 pp en el último tramo; ganancia total +17 pp entre n = 53 y n = 538; gap final 0.40	sí

3.3 Análisis cuantitativo

Tabla 3. Métricas de diagnóstico por época (PyTorch y SGD) calculadas por tracking.diagnose_epochs.

Métrica	PyTorch softmax (150 épocas)	SGD scikit-learn (80 épocas)
Diagnóstico	overfitting	overfitting
Época de mínima pérdida de validación	46	80
Pérdida de validación mínima / final	1.170 / 1.199	1.040 / 1.040
Rebote de la pérdida de validación	2.5 %	0.0 %
Pérdida de entrenamiento final	0.146	0.178
Gap de pérdida (val − train) final	1.053	0.862
Exactitud entrenamiento / validación final	1.000 / 0.556	1.000 / 0.585
Gap de exactitud final / máximo	0.444 / 0.446	0.415 / 0.446

Métrica	PyTorch softmax (150 épocas)	SGD scikit-learn (80 épocas)
F1 macro de validación final / máximo	0.501 / 0.535	0.523 / 0.555
Pendiente del gap en el último tercio (por época)	1.16e-4	-1.51e-18

Tabla 4. Gap de F1 por clase en validación cruzada por grupo (modelo base).

Clase	n entrenamiento	F1 entrenamiento	F1 validación	Gap
Pedagógico	246	0.953	0.649	0.303
Ética	219	0.956	0.649	0.307
Administrativo	100	0.932	0.360	0.571
Protección de Datos	58	0.921	0.794	0.127
Técnico	50	0.950	0.282	0.668

El gap crece a medida que disminuye el número de ejemplos de la clase: Técnico (50 ejemplos) y Administrativo (100) concentran el sobreajuste, mientras Protección de Datos, con solo 58 ejemplos pero un vocabulario muy específico, generaliza bien. El sobreajuste es, por tanto, un problema de **varianza en las clases minoritarias**, agravado por una representación con más dimensiones (5 000) que ejemplos (538).

4. Análisis de curvas de aprendizaje

4.1 Curvas por época (A y B)

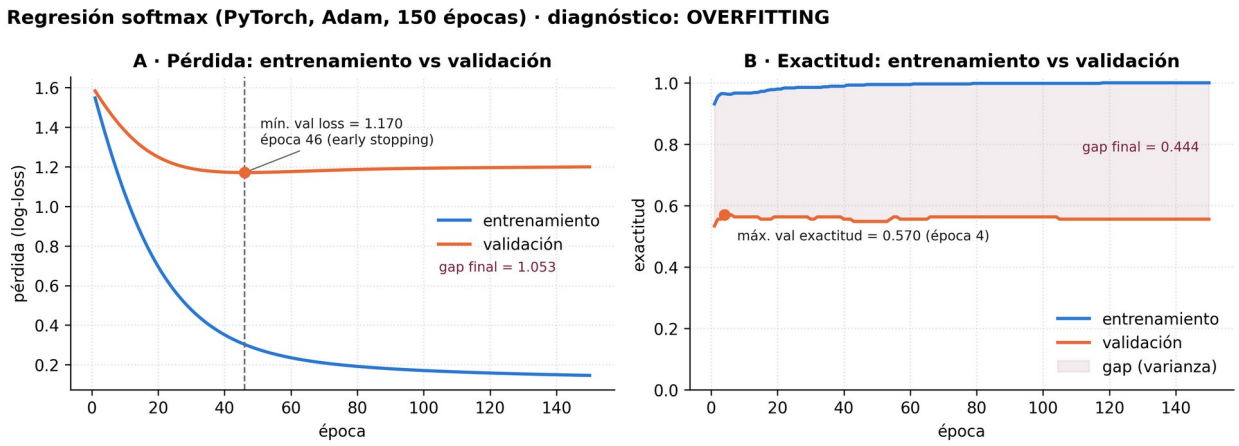


Figura 1. Regresión softmax en PyTorch (Adam, weight decay 10^{-4}, 150 épocas). Izquierda: la pérdida de entrenamiento cae de forma monótona hasta 0.15; la de validación alcanza su mínimo (1.170) en la época 46 y luego sube. Derecha: la exactitud de entrenamiento llega a 1.00 mientras la de validación se estanca en ≈ 0.56; la banda sombreada es el gap.

La curva reproduce la firma canónica del sobreajuste [16]: divergencia creciente entre pérdidas, exactitud de entrenamiento saturada y validación plana desde las primeras épocas. El punto de early stopping (época 46) está claramente definido, pero el rebote posterior es moderado (2.5 %) porque la regularización L2 y la ponderación de clases ya limitan el daño. La versión con SGD de scikit-learn (Figura 2) muestra lo mismo con otra dinámica: converge en pocas épocas y el gap se estabiliza en 0.41.

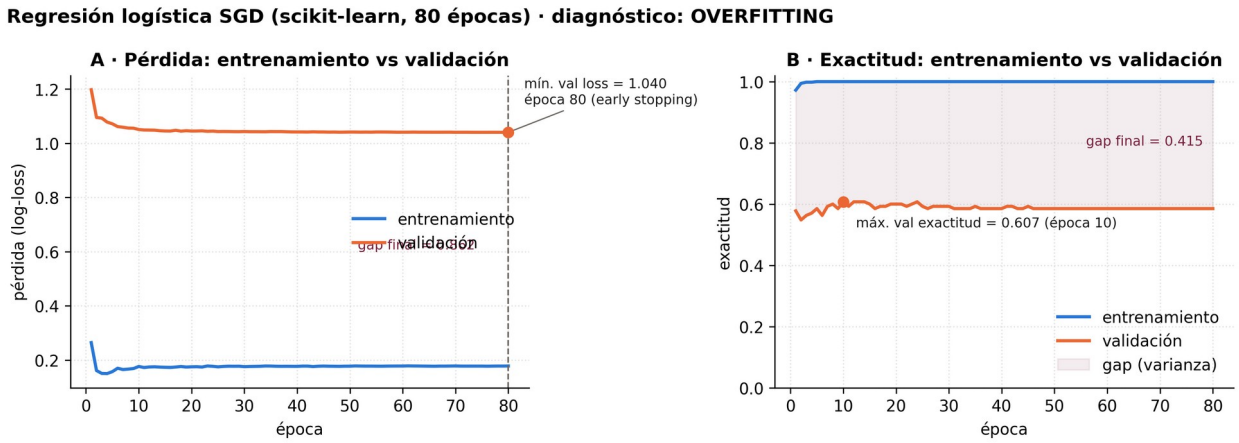


Figura 2. Regresión logística entrenada con SGD (scikit-learn, partial_fit, 80 épocas). La pérdida de entrenamiento se estabiliza en 0.18 y la de validación en 1.04; exactitud 1.00 frente a 0.59.

4.2 Curva de aprendizaje por tamaño del conjunto (C)

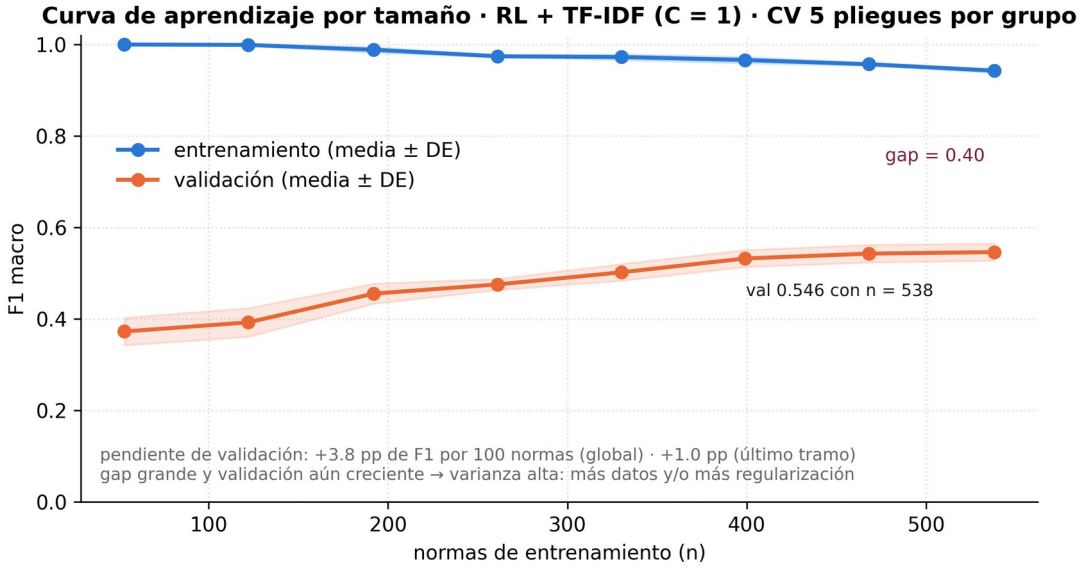


Figura 3. F1 macro frente al número de normas de entrenamiento (CV por grupo, 5 pliegues, media ± DE). La validación sube de 0.37 a 0.55; el entrenamiento baja lentamente desde 1.00; el gap final es 0.40.

Las dos curvas no convergen en el rango disponible: el gap se reduce de 0.63 a 0.40 al pasar de 53 a 538 normas, y la validación sigue subiendo, pero con rendimientos decrecientes (3.8 pp por 100 normas en promedio frente a 1.0 pp en el último tramo). En la lectura clásica de las curvas de aprendizaje [14], [15] esto indica varianza alta que más datos reducirían, pero también anticipa un techo: la extrapolación lineal del último tramo necesitaría varios cientos de normas adicionales para ganar unos pocos puntos de F1.

4.3 Curvas de validación por hiperparámetro (D)

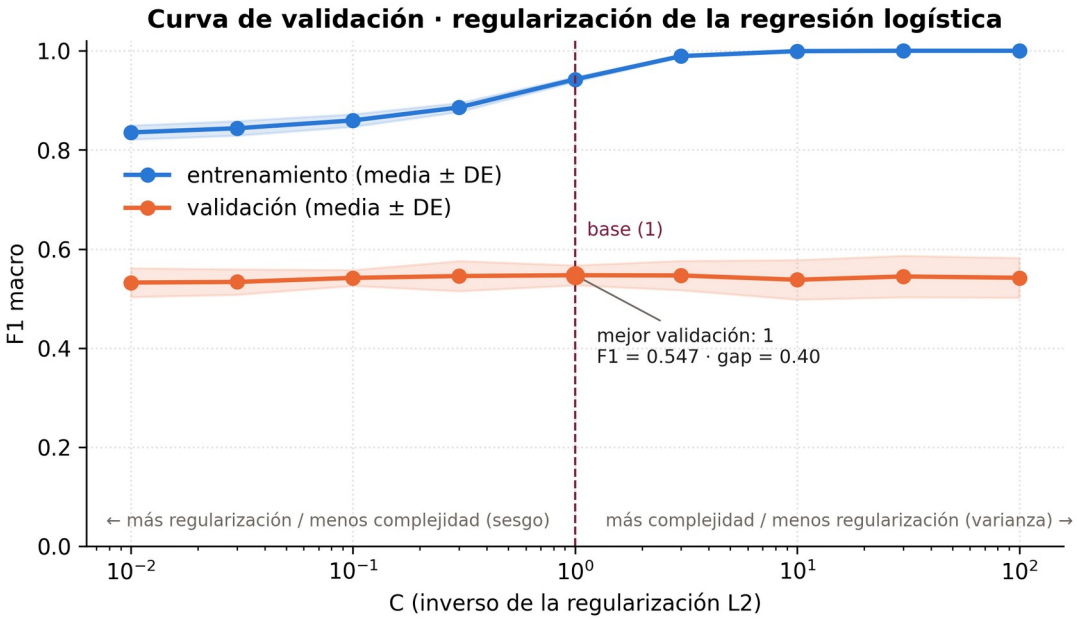


Figura 4. F1 macro frente a la fuerza de regularización C (inverso de λ en L2). El entrenamiento va de 0.84 (C = 0.01) a 1.00 (C ≥ 10); la validación se mantiene entre 0.53 y 0.55.

Esta curva es la pieza central del diagnóstico. En un sobreajuste «puro» la validación debería mejorar al aumentar la regularización hasta un óptimo y luego caer por sesgo; aquí es **plana**: la regularización controla cuánto memoriza el modelo (eje de la varianza) sin cambiar lo que acierta en validación. Eso significa que una fracción sustancial del gap no es varianza sino **error irreducible** del problema —normas en la frontera entre dos dimensiones, con una sola etiqueta por norma— y posible ruido de etiquetado [17], [18], consistente con el hallazgo de la Semana 2: el modelo falla donde el codificador humano declaró confianza media y donde asignó etiquetas secundarias.

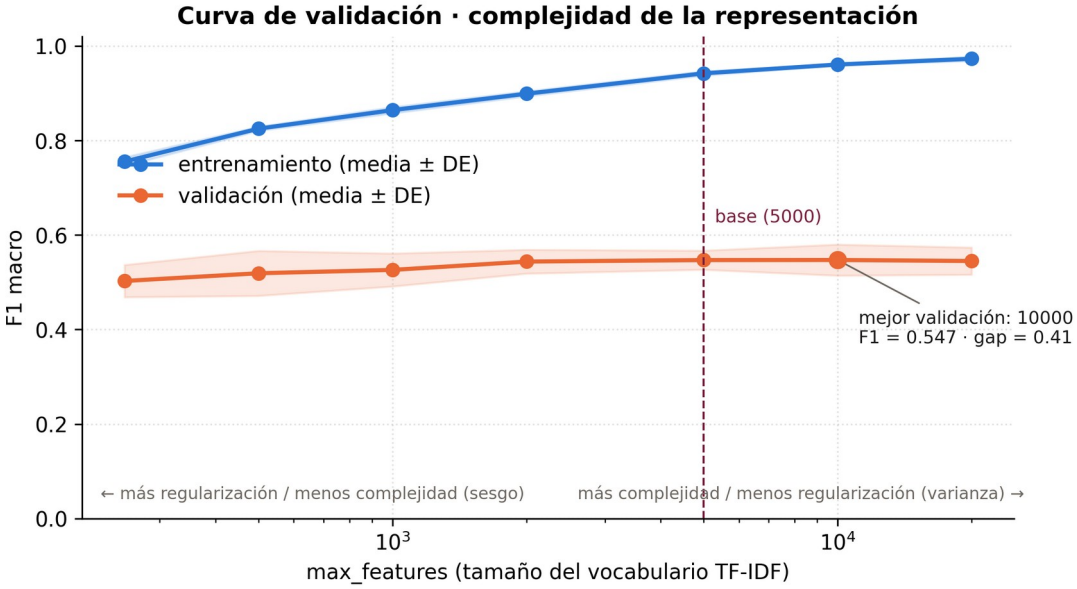


Figura 5. F1 macro frente al tamaño del vocabulario TF-IDF. El entrenamiento crece con la complejidad de la representación; la validación es prácticamente constante a partir de 2 000 características.

La curva de complejidad confirma la lectura: a partir de ≈2 000 características la validación no gana nada, mientras el entrenamiento sigue memorizando. Un vocabulario de 2 000 términos es, por tanto, suficiente y reduce la varianza sin costo [19].

5. Estrategias implementadas y resultados

El diagnóstico es overfitting, así que se implementaron las cuatro estrategias de la lista de la actividad que aplican a un modelo lineal y se justificó la no implementación de las restantes (Tabla 6).

Tabla 5. Decisión sobre cada estrategia recomendada para overfitting.

Estrategia	Decisión	Justificación
E1 · Regularización (L2)	implementada	barrido de C (curva D) con regla de 1 DE; barrido de weight decay en PyTorch
E2 · Early stopping	implementada	paciencia 15 épocas sobre la pérdida de validación; restaura la mejor época [13]
E3 · Reducción de complejidad	implementada	vocabulario de 1 000-2 000 términos, min_df = 2, unigramas
E4 · Validación cruzada más rigurosa	implementada	CV por grupo de URL frente a CV ingenua; cuantifica la fuga por duplicados
Aumento de datos	no implementada	no hay normas nuevas disponibles; la retrotraducción requiere API externa y altera el léxico jurídico que es la señal; sustituir sinónimos en textos de 44 palabras cruza de clase («protección de datos» ↔ «privacidad»); el sobremuestreo equivale a class_weight='balanced', ya activo
Dropout	no aplica	solo tiene sentido con capas ocultas [20]; el modelo es lineal por decisión de interpretabilidad (Ficha de Decisión Técnica); train_torch_softmax(hidden>0) deja implementado un MLP con Dropout para trabajo futuro

5.1 E1 · Regularización

La regla de una desviación estándar sobre la curva D selecciona C = 0.01 (el máximo de validación, 0.547, se alcanza en C = 1 con DE 0.020; C = 0.01 está dentro de esa banda con F1 0.532). El gap de F1 baja de 0.40 a 0.30 con una pérdida de validación de 0.015, inferior a la desviación entre pliegues. En PyTorch, el barrido de weight decay (Tabla 7) muestra el mismo patrón con más claridad: la penalización L2 más fuerte reduce el gap de exactitud de 0.44 a 0.25 y eleva el F1 máximo de validación de 0.535 a 0.582 (máximo sobre épocas en una sola partición: estimación optimista, útil para comparar configuraciones).

Tabla 6. Barrido de weight decay (L2) en el bucle de PyTorch, partición 80/20, 150 épocas.

weight decay	Exactitud entrenamiento	Exactitud validación	Gap	F1 val. máximo	Pérdida val. mínima	Rebote
0.0001	1.000	0.556	0.444	0.535	1.170	2.5 %
0.001	0.985	0.607	0.378	0.548	1.189	0.0 %
0.01	0.890	0.585	0.305	0.567	1.405	0.0 %
0.05	0.861	0.607	0.253	0.582	1.543	0.2 %

La pérdida de validación mínima sube con el weight decay porque la penalización encoge los logits y produce probabilidades menos extremas; la exactitud y el F1, que dependen solo del argmax, mejoran. Es un recordatorio de que la pérdida y las métricas de decisión pueden moverse en sentidos opuestos y de que el diagnóstico debe mirar ambas.

5.2 E2 · Early stopping

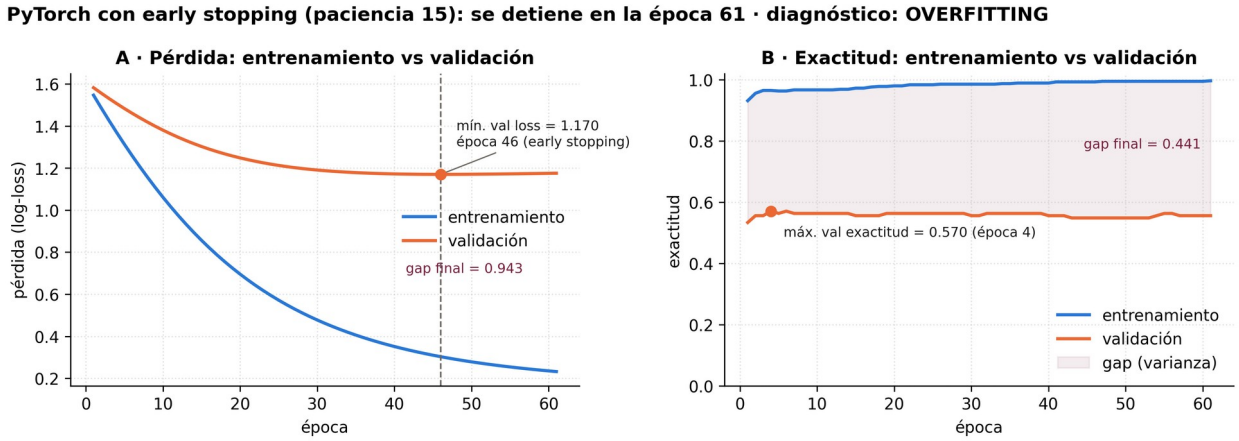


Figura 6. Entrenamiento en PyTorch con paciencia de 15 épocas: se detiene en la época 46 y restaura los pesos de la mejor época.

Tabla 7. Efecto del early stopping (PyTorch, partición 80/20).

Configuración	Épocas	Pérdida val.	Exactitud val.	F1 val.	Exactitud entren.	Gap exactitud
150 épocas (sin early stopping)	150	1.199	0.556	0.501	1.000	0.444
early stopping (paciencia 15) → época 46	46	1.170	0.548	0.505	0.993	0.444

El early stopping elimina el rebote de la pérdida de validación y ahorra 104 épocas de cómputo con el mismo F1 (0.505 frente a 0.501). No reduce el gap de exactitud porque el modelo ya ha memorizado el entrenamiento en la época 46: es una salvaguarda contra el deterioro, no un sustituto de la regularización.

5.3 E3 · Reducción de la complejidad de la representación

Tabla 8. Variantes de la representación TF-IDF (CV por grupo, 5 pliegues).

Configuración	F1 entrenamiento	F1 validación	DE val.	Gap
max_features = 1000	0.864	0.526	0.039	0.338
max_features = 2000, min_df = 2	0.901	0.539	0.027	0.362
unigramas, max_features = 2000	0.918	0.551	0.024	0.366
unigramas, min_df = 2, C = 0.01	0.756	0.518	0.044	0.238

5.4 E4 · Validación cruzada más rigurosa

La diferencia entre la validación cruzada ingenua (*StratifiedKFold*) y la validación por grupo de URL es de -0.003 de F1 macro: en este subconjunto solo hay 8 grupos de URL repetida, así que la fuga es despreciable. La CV por grupo se mantiene en el protocolo porque no tiene costo y protege la evaluación cuando el corpus crezca (el corpus completo tiene 14 grupos, 8 de ellos con dimensiones distintas). El resultado es, en sí, un hallazgo: descarta la fuga de duplicados como explicación del gap.

5.5 Comparación antes/después

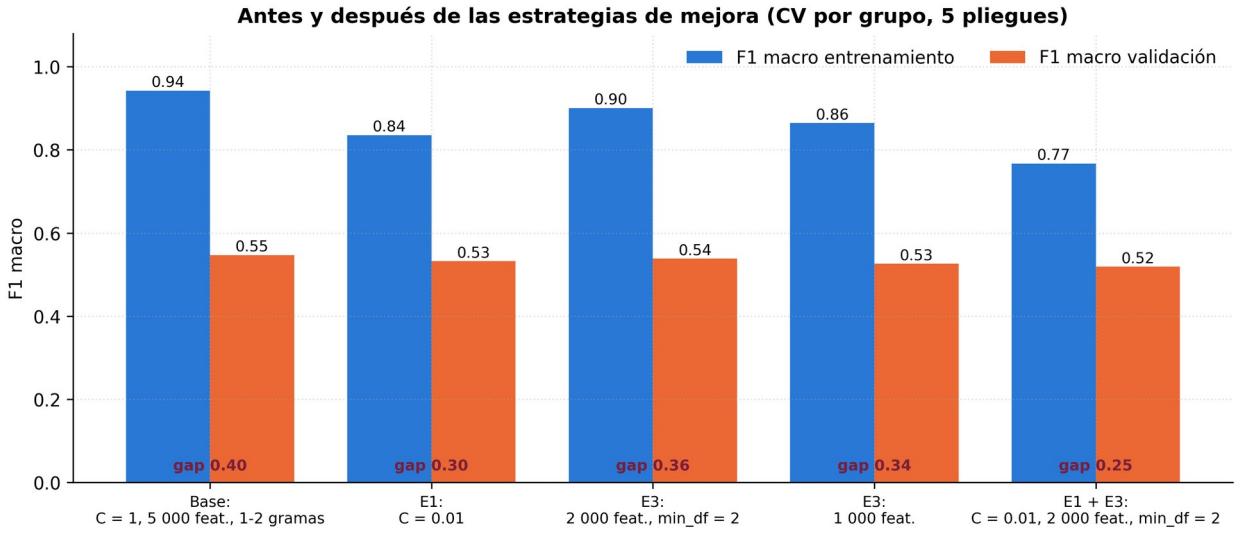


Figura 7. F1 macro de entrenamiento y validación por configuración (CV por grupo, 5 pliegues); el gap se anota en la base de cada par.

Tabla 9. Comparación antes/después de las estrategias (CV por grupo, 5 pliegues, mismas particiones).

Configuración	F1 entren.	F1 val.	DE val.	Gap	Δ F1 val.	Δ gap
Base: C = 1, 5 000 feat., 1-2 gramas	0.942	0.547	0.023	0.395	+0.000	+0.000
E1: C = 0.01	0.835	0.532	0.033	0.303	-0.015	-0.092
E3: 2 000 feat., min_df = 2	0.901	0.539	0.027	0.362	-0.008	-0.033
E3: 1 000 feat.	0.864	0.526	0.039	0.338	-0.021	-0.057
E1 + E3: C = 0.01, 2 000 feat., min_df = 2	0.767	0.519	0.019	0.248	-0.028	-0.147

Las estrategias cumplen su propósito —reducir la varianza—: la configuración combinada (C = 0.01, 2 000 características, min_df = 2) baja el gap de 0.40 a 0.25, un 37 % menos, con un F1 de validación 0.519 que difiere de la base en -0.028, del orden de la desviación entre pliegues. Ninguna estrategia mejora significativamente la validación: es exactamente lo que predice la curva de validación plana por C. Para el proyecto, la configuración regularizada es preferible al mismo rendimiento porque sus probabilidades son menos extremas (mejor comportamiento del umbral de confianza del flujo operativo) y porque es menos sensible a la muestra.

6. Conclusiones y recomendaciones futuras

1. **El modelo sobreajusta y el diagnóstico es robusto a la herramienta.** Con tres implementaciones (L-BFGS, SGD, PyTorch) y tres tipos de curva, el veredicto coincide: exactitud de entrenamiento ≈ 1.00, validación ≈ 0.56, gap de F1 0.40, mínimo de la pérdida de validación en la época 46. No hay underfitting.
2. **El sobreajuste tiene dos componentes.** Uno reducible —varianza por 5 000 características para 538 ejemplos y por las clases minoritarias—, que la regularización y la reducción del vocabulario recortan (gap 0.40 → 0.25); y otro irreducible —normas de frontera y etiqueta única—, que ninguna regularización mueve, como muestra la curva de validación plana por C.
3. **Más épocas no ayudan; más datos ayudan con rendimientos decrecientes.** La curva por tamaño gana 17 pp de F1 entre 53 y 538 normas, pero solo 1.0 pp por 100 normas en el último tramo: el retorno está en codificar más normas de las clases Técnico y Administrativo, no en más datos del mismo perfil.
4. **Recomendaciones para el Proyecto Integrador.** (i) Adoptar la configuración regularizada (C bajo, 2 000 características) como modelo clásico de referencia: mismo rendimiento, menor varianza, probabilidades más conservadoras para el umbral de confianza. (ii) Mantener la CV por grupo y el early stopping en cualquier modelo entrenado por épocas. (iii) Priorizar la rama LLM few-shot y los embeddings

preentrenados, que aportan conocimiento externo donde el modelo lineal ya no puede extraer más señal del texto. (iv) Usar la hoja de tipificación de errores para revisar las etiquetas ambiguas: reducir el ruido de etiquetado es la forma más directa de bajar el error irreducible [17]. (v) Informar siempre entrenamiento y validación, con intervalos, y fijar los umbrales de diagnóstico antes de mirar las curvas.

5. **Reflexión.** El tracking por época hizo visible lo que el punto final de scikit-learn ocultaba (el rebote de la pérdida de validación), y la curva de validación por hiperparámetro corrigió la hipótesis inicial de que bastaría con regularizar: el diagnóstico sistemático cambió la estrategia de mejora del proyecto, de «ajustar el modelo» a «mejorar los datos y la representación».

7. Referencias técnicas

[1] G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” Journal of Machine Learning Research, vol. 11, pp. 2079–2107, 2010.

[2] S. Raschka, “Model evaluation, model selection, and algorithm selection in machine learning,” arXiv:1811.12808, 2018.

[3] scikit-learn developers, “Stochastic Gradient Descent,” scikit-learn 1.8 documentation, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/sgd.html (accedido: 1 oct. 2026).

[4] L. Bottou, “Large-scale machine learning with stochastic gradient descent,” in Proc. COMPSTAT 2010, Paris, France, 2010, pp. 177–186, doi: 10.1007/978-3-7908-2604-3_16.

[5] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32 (NeurIPS), Vancouver, Canada, 2019, pp. 8024–8035.

[6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. 3rd Int. Conf. on Learning Representations (ICLR), San Diego, CA, USA, 2015.

[7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] scikit-learn developers, “Validation curves: plotting scores to evaluate models,” scikit-learn 1.8 documentation, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/learning_curve.html (accedido: 20 sep. 2026).

[9] MLflow Project, “MLflow Tracking,” MLflow documentation, 2026. [Online]. Available: https://mlflow.org/docs/latest/tracking.html (accedido: 1 oct. 2026).

[10] S. Geman, E. Bienenstock, and R. Doursat, “Neural networks and the bias/variance dilemma,” Neural Computation, vol. 4, no. 1, pp. 1–58, 1992, doi: 10.1162/neco.1992.4.1.1.

[11] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006, cap. 3.2 (descomposición sesgo-varianza).

[12] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009, caps. 7 (Model Assessment and Selection) y 4.4.

[13] L. Prechelt, “Early stopping — but when?,” in Neural Networks: Tricks of the Trade, G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer, 1998, pp. 55–69, doi: 10.1007/3-540-49430-8_3.

[14] C. Perlich, F. Provost, and J. S. Simonoff, “Tree induction vs. logistic regression: A learning-curve analysis,” Journal of Machine Learning Research, vol. 4, pp. 211–255, 2003.

[15] F. Mohr and J. N. van Rijn, “Learning curves for decision making in supervised machine learning: A survey,” Machine Learning, vol. 113, pp. 8371–8425, 2024, doi: 10.1007/s10994-024-06619-7.

[16] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016, caps. 5 (capacidad, sobreajuste y subajuste) y 7 (regularización, early stopping).

[17] B. Frénay and M. Verleysen, “Classification in the presence of label noise: A survey,” IEEE Transactions on Neural Networks and Learning Systems, vol. 25, no. 5, pp. 845–869, 2014, doi: 10.1109/TNNLS.2013.2292894.

[18] P. Domingos, “A few useful things to know about machine learning,” Communications of the ACM, vol. 55, no. 10, pp. 78–87, 2012, doi: 10.1145/2347736.2347755.

[19] A. Y. Ng, “Feature selection, L1 vs. L2 regularization, and rotational invariance,” in Proc. 21st Int. Conf. on Machine Learning (ICML), Banff, Canada, 2004, doi: 10.1145/1015330.1015435.

[20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

Anexo. Archivos entregados

Archivo	Contenido
notebooks/overfitting_analysis.ipynb (+ .html)	cuaderno ejecutado: carga, tracking, modelo base con logging, curvas A-D, diagnóstico, estrategias E1-E4, comparación antes/después, conclusiones
src/genial_clf/tracking.py	código modular: MetricsTracker, EpochTrainer (scikit-learn), train_torch_softmax (PyTorch), KerasTrackerCallback, curvas con CV por grupo, diagnose_epochs / diagnose_size, figuras
results/figuras/overfitting/01-08*.png	ocho figuras a 300 DPI
results/overfitting/*.csv · *.json	historiales por época (SGD, PyTorch, early stopping, weight decay), curvas, tablas de diagnóstico y comparación, síntesis (sintesis_overfitting.json) de la que se toman todas las cifras de este reporte
docs/semana3/diagnostic_report.pdf	este reporte